

HW

Stile Notes

02. FAST API

FAST API

Building APIs with Python 3.7 +
standards OpenAPI (Swagger) & JSON
User static Typing

SQLAlchemy

- is an ORM
- Classes operating as objects
- Interactive with databases
- Productive backends.

2.07.00F Fundation of User Interface Design.

Human Interface

Four laws

1. Not harm your work
2. Not waste your time
3. It's humane if it is responsive
4. Set the pace of interaction

2.07.01 Cognates And locus of Attention

HCI Constants (Human limitation)

- locus of Attention
- Cost of shifting
- Cognitions

Design Implications (Raskin's Rules)

- Don't interrupt: Interfaces should allow users to concentrate solely on their job
- Assumption of Absorption: Design as if the user is so absorbed in their task that they will not respond to system attempts to communicate

Main Takeaway: Good UI design must command the fact that user cannot multitask cognitively; they focus on one thing at a time.

2.07.01 Getting started with UI design

The design process

User analysis: Understand what the users will do with the system

System prototyping: Create early versions for experimentation

Interface Evaluation: Test prototypes with users.

Key design principles

User familiarity: Use terms and concepts from the user's experience rather than computer jargon.

Consistency: Similar operations should be activated in the same way

Minimal Surprise: System behavior should not surprise the user

Recoverability: Include: "Undo" functions and confirmations for destructive actions.

User Guidance: Provide meaningful feedback and context-sensitive help

User Diversity: Design for both casual users and power users.

User interface models

- Design Model: How the engineer views the system
- User Model: The profile of the users.
- System Image: How the user thinks the system works.

Software Development Models

- Waterfall (SDLC)
- Agile (SCRUM) → short sprints, iterative delivery.
- Extreme Programming (Xp) → user stories, pair programming, continuous integrations.

User Research

- Methods: observation, interviews, surveys, personas.
- Focus: users goals, tasks, vocabulary, skills, attitudes, past experiences.

UI design Challenges

Require understanding users, workflows, expectations

Increasing complexity: diverse devices, higher user expectations, broader user base.

UI metaphors & Idioms

Metaphors: desktop, control panel, store, art room.

Idioms: forms, text, editors, spreadsheets, browsers, media players, immersive games.

2.07.02 Information Organization

Determining Content

Identify actors and user goals

Analyze objects, relationships, and requirements

Explore real scenarios (use case, diagrams, state charts)

Dividing Content

Break scenarios into logical chunks of interaction

Keep chunks cohesive and loosely coupled

Use classification/generalization to simplify

UI Metaphors

Desktop, control panel, store/shopping, art room.

Metaphors help bridge user familiarity with system design.

Interface idioms

Recognizable styles: forms, text editors, spreadsheets, browsers, calendars, media players, games, e-commerce sites.

Layout & Structures

Organize collections: objects, tasks, tools.

Physical windows, one-windows paging, tiled panes.

Design Patterns

Proven solutions to recurring problems

Context (Why), problem (When), solution (What/How)

UI Organizational Patterns

Information patterns: Search and browse, New Stream, Picture Manager, Dashboard, Settings Editor, Many workspaces.

Windows structure patterns: Two panel selector, Canvas + Palette, One-window drilldown, Alternative views.

Others patterns: Wizard, Extras on demand, Intriguing branches, Multi-level help.

27.07.03 Page Layout

Visual Hierarchy

Goal: Organize content so users know where to look first.

Tools: Use size, color, contrast, and positions to distinguish important elements from secondary ones.

The Squint Test: If your eyes (blurring the details), the most important elements should still stand out.

Gestalt Principles (How we perceive groups)

Proximity

Similarity

Closure

Continuity

List & Structures

Organize collections: objects, tasks, tools.

Physicals windows structures: multiple windows, one-window paging, tiled panes.

Design Patterns

Proven solutions to recurring problems

Context (why), problem (When), solution (What / how)

2.07.04 Requirements Definition

Context & purpose

T light: tool for modeling and experimenting with traffic flow.

Users: cartographers (map creation), traffic engineers (simulation design), analyst (data analysis), administrator (system management)

User goals

Cartographers → create, edit, delete, print maps.

Traffic engineers → design simulation, set vehicle rates, turning probabilities, run/save simulations.

Analysts → open/run datasets, view graphs, perform static analysis.

Administrators → manage user accounts and system configuration.

Functional requirements

Authentication & access control

Map Building

Simulation Features

Analysis Features

Administration Features.

Non-Functional Requirements.

Incremental development process

testing with NUnit and NUnitForms

2.07.06 Slides Navigation

Four Key Questions

Where am I now?

Where do I want to go?

How do I get there?

Am I there yet?

Signposts.

Help user stay oriented

Examples: windows titles, logos, branding, highlighted tabs, selection indicators.

You are here Patterns

Sequence Map

Bread crumbs

Annotated Scrollbar

Color-coded sections.

Wayfinding

User finding their way toward goals

Aids: good labels, signage, environmental clues, maps, directions.

"Wayshowing" designers guiding users effectively

Navigational Models.

Hub-and-spoke

freely connected

Multi-level

Book Marks

Step curve

Pyramid

Pan-and-zoom

Escape hatch

flat navigation

Modal panel

Clear entry points

Navigation Model Patterns

Clear entry points → guide first-time users

Menu page → table of contents style

Pyramid → sequence of page, not strictly ordered

Modal panel → force user action before continuing

Deep/linked state → capture app/site state for return

Escape hatch → way out of a process.

Layout + Model Patterns

Fat menus (long dropdowns)

Site map footer

Pagination (Google-style)

Sign-in tools.

2.07.09 Actions And Commands

Common Sense Principles:

Follow established GUI conventions when appropriate

Make desired actions visible, well-labeled, and easy to find

Support sequences, automations, and repetition

Action/Command Mechanisms

Visible → buttons, menus, toolbars, links, action panels

Invisible → clicks, keyboard, shortcuts, drag-and-drop, typed commands

Action / Command Patterns

Button groups

Hover tools

Action panels

Prominent "Done" button

Multi-level undo & command history

Intelligence

Smart menu items

Preview

Progress indicators

Connectability

Macros

Multiple gesture for one action /
one gesture triggering multiple actions.

2.07.10 Showing Complex Data

Information Graphics

Present data visually to impart knowledge

Tools: graphics, tables, trees, maps.

Spatial relationships convey meaning

Development Activities

User research → abilities, priorities, attitudes

System Analysis → real-world problems, available data.

Organizational Models

Linear (list, single-variable plots).

Tubular (spreadsheets, sortable tables)

Hierarchical (trees, cascaded lists)

Network (graphs, flow charts)

Geographics/spatial (maps, schematics).

Shape & Appearance

Shape conveys meaning

Similar data $\rightarrow$ consistent look

Important items $\rightarrow$ stand out

Navigation & Browsing

Scroll, pan, zoom

Drill down, magnify, open/close points of interest

Sorting & Rearranging

Alphabetical, numerical, date/time, location, category, popularity, user-defined random.

Filtering & Querying

Best when interactive, iterative, contextual

Giving Data Meaning

Labels, legends, axes, scales, timelines

Datatypes, data brushing

Patterns

Iterative graphics $\rightarrow$ overview + detail, dynamic queries, local zooming

Table/List $\rightarrow$ row striping, sortable tables, jump to item, new-item row

Hierarchical data $\rightarrow$ cascading list, tree tables

Multidimensional data $\rightarrow$ multi-Y graphs, small multiples, treemaps

2-07-11 Getting Input from the user

Principles

Make questions clear and necessary

Prefer Knowledge "in the world" over "in the head"

Usability testing is essential

Control Choice Factors

Space available, user skill, domain knowledge, expectations, technology

Type of controls

Binary choices → check box, radio buttons, toggles.

Small sets (N items) → radio buttons, dropdowns, spinners.

Large sets → add/new buttons, drag-and-drop

Ordering → up/down buttons, drag-and-drop.

Text input → single/multi-line boxes, rich text, autocomplete

numbers/ranges → sliders, spinners, validated, text boxes

Dates/times → structured text, calendars, pickers.

Patterns

Text entry → forgiving formats, prompts, autocomplete

choice → dropdowns, illustrated options, list builders.

Other → good defaults, inline error message.